



基于LLM的DSL设计与漏洞检测

北京大学软件安全研究团队

北京大学软件与微电子学院
北京大学网络与软件安全保障实验室

目录

1 / 背景介绍

2 / DSL-Xpert: LLM 驱动的通用 DSL 代码生成

3 / DSL Assistant 从自然语言到正式语言

4 / MoCQ:通过整体神经符号方法自动检测静态漏洞

5 / 领域驱动的 LLM 开发：对 RAG 和微调实践的见解

Part01

背景介绍



北京大学

PEKING UNIVERSITY

北京大学软件与微电子学院
School of Software & Microelectronics

INFOSEC

Laboratory

北京大学信息安全实验室

网络和软件安全保障教育部重点实验室



小组主页: <http://www.pku-exploit.com/>

● 现有漏洞检测方法的困难点

- **人工依赖与高成本：**

传统静态分析依赖专家手动编写检测规则（如CodeQL查询），耗时长且易出错。

- **精度与泛化的平衡困境：**

LLM直接检测漏洞时，缺乏跨文件分析能力，上下文理解有限

- **工具适配性差：**

主流工具（如CodeQL）不支持部分语言（如PHP），且动态特性（如JavaScript原型链）难建模

● 利用DSL(领域特定语言,domain specific language)进行漏洞检测的优势

- **精准表达领域逻辑：**

DSL通过中间表示（IR）可以直接映射漏洞模式，减少无用信息

- **降低领域知识门槛：**

DSL利用IR信息可以适配所有自然语言表示，无需掌握不同语言的语法

- **提升自动化效率：**

DSL简化漏洞检测规则的开发和维护。其语法精简且领域聚焦，使开发者能快速编写、迭代和复用查询

Part02

DSL-Xpert: LLM 驱动的 通用 DSL 代码生成(2024)

<https://dl.acm.org/doi/abs/10.1145/3652620.3687782>



北京大学

PEKING UNIVERSITY

北京大学软件与微电子学院
School of Software & Microelectronics

INFOSEC

Laboratory

北京大学信息安全实验室

网络和软件安全保障教育部重点实验室



小组主页: <http://www.pku-exploit.com/>

DSLs的重要性与挑战

• **核心作用**：在软件工程领域，DSL是表达和管理特定领域概念的关键工具。与通用编程语言不同，DSLs 针对特定领域需求设计，提供更高的抽象性、表达力和效率，尤其适用于非专业开发人员。它们优化了工作流，增强了软件的可重用性、可维护性和可扩展性。

• 主要痛点：

• 开发成本高：

设计、实现和维护每个DSL需要专业知识，导致开发周期长

• 学习曲线陡峭：

缺乏完善的工具支持和文档增加了学习难度

• 普及障碍

许多DSL是私有或未发布的，通用工具难以直接适配
开发者需投入额外精力掌握特定语言

Create a sensor called 'Traffic' with an interval of 300, made in postgres and a point geometry. It has a series of properties defined below: desc (String) and name (String). As for the measurement data, they are presented below, together with their typology and units in brackets: intensity (Double, 'veh/h'), speed (Double, 'km/h'), occupancy (Double, '%'). Finally, this sensor belongs to two different spatial dimensions: one called 'Administration' which includes 'Section', 'District' and 'Municipality'; and another called 'Neighbourhood' which includes the dimension called 'Neighbourhood'.

```
CREATE SENSOR Traffic (  
  interval: 300,  
  datasource: postgres,  
  geometry: Point  
) WITH PROPERTIES (  
  desc String,  
  name String  
) WITH MEASUREMENT DATA (  
  intensity Double UNITS "veh/h",  
  speed Double UNITS "km/h",  
  occupancy Double UNITS "%"  
) WITH SPATIAL DIMENSIONS Administration (  
  Section, District, Municipality  
) WITH SPATIAL DIMENSIONS Neighbourhood (  
  Neighbourhood );
```

2. LLMs的潜力与局限

- **机遇**: LLMs (如GPT-3/4) 的出现为DSL工程带来变革。这些模型基于海量自然语言数据训练, 能理解复杂语义和模式, 可用于自动化DSL代码生成、文档编写等任务, 显著提升开发效率。
- **关键局限**: LLMs在处理未公开或冷门DSLs时效果不佳。
 - **训练数据偏差**: 模型未接触过小众DSL词汇, 导致语义解析错误。
 - **通用性不足**: LLM擅长常见任务 (如SQL), 但对特定DSL的代码生成缺乏针对性支持。

定义

语义解析利用LLM理解自然语言指令的意图。

例如，当用户输入“create a new product”时

LLM不直接执行“创建新产品”

而是将其翻译为DSL词汇，CREATE PRODUCT。

LLM的适配性

预训练模型（如GPT、LLaMA）已具备强大的语言理解能力
无需从头训练定制模型。

局限性：

通用LLMs对未训练过的DSL词汇识别率低，需额外约束机制。

核心原理

将DSL语法规则作为上下文注入LLM，限制输出范围。

例如，通过BNF (*Backus-Naur Form*) 明确定义语法结构：

`createProduct ::= "PRODUCT" identifier "USING" intNumber ";"`;

避免LLM生成无效符号

实现方式

- BNF可以提供清晰的语法树结构
- JSON/XML格式

如 `{"command": "CREATE", "type": "PRODUCT"}`
也可实现类似效果。

```
root ::= createStatement+
createStatement ::= "CREATE " (createProduct |
    createRange | createDimension | createSensor)
createProduct ::= "PRODUCT " identifier "USING"
    intNumber ";"
createRange ::= "RANGE " identifier "(" rangeProp (","
    rangeProp)* ")"
rangeProp ::= rangeNumber ("TO " rangeNumber)? "AS "
    identifier ("COLOR " hexColor)? | "DEFAULT " "AS "
    identifier ("COLOR " hexColor)?
rangeNumber ::= identifier | floatNumber | intNumber |
    "INFINITY" | "-INFINITY"
createDimension ::= "SPATIAL DIMENSION " identifier "("
    "geometry" ":" (type | "Geometry") ")"
    createDimProp ";"
createDimProp ::= "WITH PROPERTIES (" dimPropDef (","
    dimPropDef)* ")"
dimPropDef ::= identifier type
createSensor ::= "SENSOR " identifier "(" "interval" ":"
    intNumber "," "datasource" ":" dataSource ","
    "geometry" ":" type ")" createSensorProp
    sensorMeasData (sensorSpatDim)* ";"
createSensorProp ::= "WITH PROPERTIES (" sensorProp (","
    sensorProp)* ")"
```

```
Create a range called MagnitudRange with the following  
definitions: 0-3 for surface , 4.75-5.25 for 5m and  
9.75-10.25 for 10m.
```

```
CREATE RANGE MagnitudRange (0 TO 3 AS "surface",4.75 TO  
5.25 AS "5m",9.75 TO 10.25 AS "10m");
```

DSL学习示例

核心原理

提供少量输入-输出示例，引导LLM学习模式

例如：

自然语言指令： *"Create a product called intecmar with SRID 4326"*

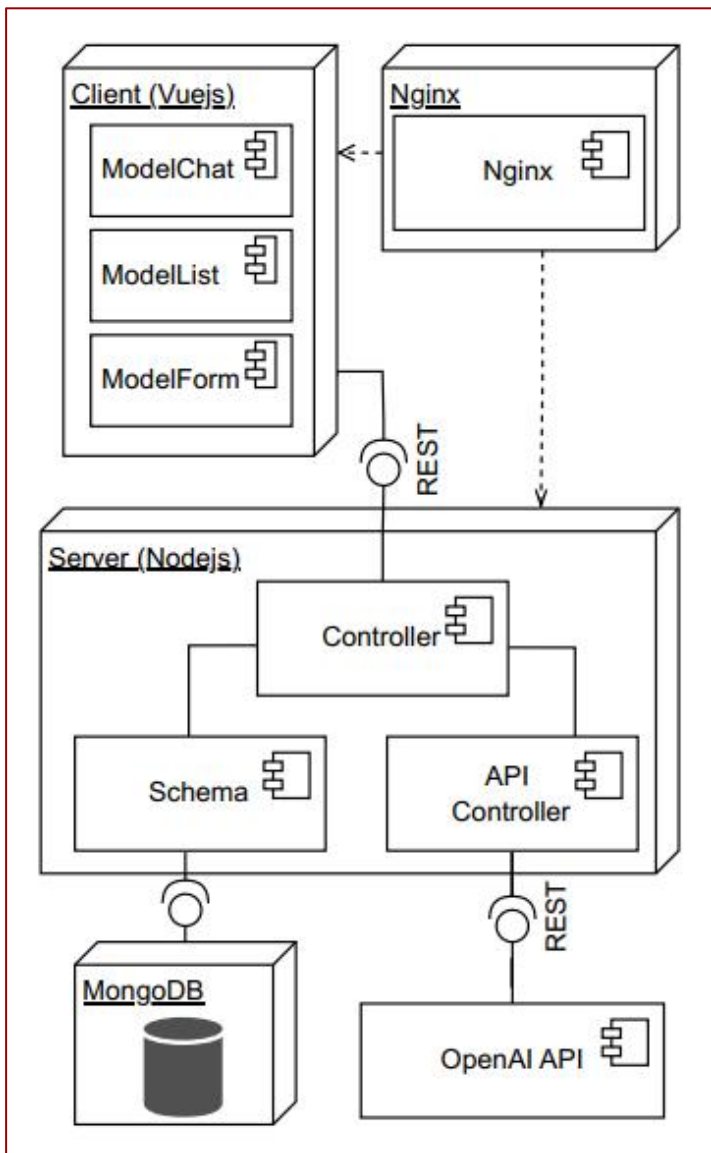
DSL输出： *`CREATE PRODUCT intecmar USING 4326;`*

优势

能够从一系列以前的使用示例中生成新的实例，吸收和检测这些示例中的公共模式

局限

依赖示例质量，若新指令与示例差异过大（如跨领域操作），准确率显著下降。



系统架构图

•Client:

基于Vue 3和Vuetify构建的用户界面
使用Vue Router管理导航。

•Server:

Node.js和Express.js实现的RESTful API
处理模型CRUD和聊天请求
并通过Mongoose连接MongoDB。

•Database

MongoDB存储模型配置和聊天历史。

•Dockerization:

各组件 (Client、Server、DB、Nginx) 容器化
由Docker Compose统一管理，简化部署。

Nginx处理路由，确保单一入口点
(localhost:5555)。

•优势:

- **开发效率**: 减少DSL学习曲线, 非专家用户可快速生成代码。
- **资源高效**: 避免LLM微调, 仅需提示配置。
- **泛化性**: 支持任意DSL, 通过模块化模型适应多领域。

•局限:

- **依赖语法和示例质量**: 不全面示例可能导致错误输出。
- **通用性限制**: 复杂DSL需定制配置。
- **未进行严格评估**: 需更多真实场景测试。

Part03

DSL Assistant 从自然语言到正式语言(2024)

<https://dl.acm.org/doi/abs/10.1145/3652620.3687811>



北京大学

PEKING UNIVERSITY

北京大学软件与微电子学院
School of Software & Microelectronics

INFOSEC

Laboratory

北京大学信息安全实验室

网络和软件安全保障教育部重点实验室



小组主页: <http://www.pku-exploit.com/>

1. 领域特定语言 (DSL) 开发的挑战

1. DSL设计是**模型驱动工程 (MDE)** 的核心环节, 旨在为特定领域专家提供高抽象级、低复杂度的专用语言 (对比通用编程语言GPL)。
2. 传统开发过程存在痛点:
 1. **劳动密集型迭代**: 需反复定义语法、语义及实例。
 2. **专业知识鸿沟**: 领域专家缺乏编程技能, 难独立完成DSL开发。
 3. **错误修复成本高**: 手动调试语法错误效率低下。

2. 生成式AI的兴起

1. 大型语言模型 (LLM) 如GPT-4o在**代码生成** (如GitHub Copilot) 和**自然语言理解**方面展现突破, 为弥合领域专家与开发者间的沟通鸿沟提供可能。
2. 现有局限: LLM在MDE任务中仍存在**语法/语义缺陷**和**响应不一致**问题

三个独立决策维度：

- 操作类型 (Kind)

DSL (定义语法)

Example (生成实例)

- 输入格式 (Input Format)

Formal Definition (语法/实例文本)

Properties (自然语言描述)

Error Message (Xtext错误反馈)

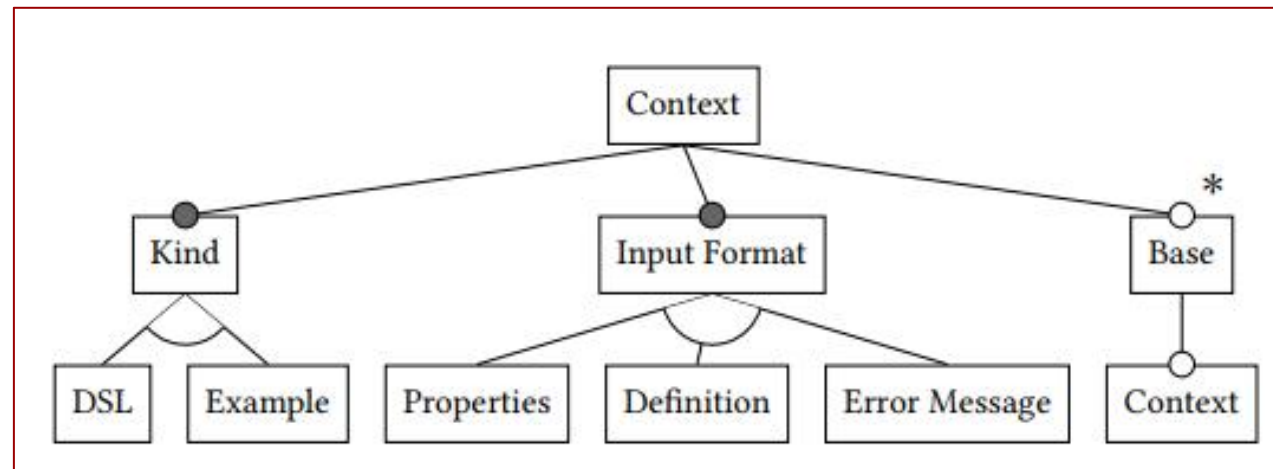
- 基版本状态 (Base)

无基版本 (新建)

有基版本 (迭代)

四种约束规则：

- 1.多基版本条件：仅当泛化多个实例时允许base>1
- 2.错误修复依赖：错误输入必须关联故障
- 3.基版本类型一致性：新版本与基版本类型必须相同最新
- 4.版本约束：基版本必须无后继版本



基于特征模型的LLM上下文

有效性证明：

维度完备性：

三维度覆盖DSL开发全场景（新建/扩展/修复/实例化）

约束兼容性：

四项约束仅限定配置的使用条件（如错误修复需基版本）

但不减少组合数

工程可实现性：所有配置均通过特征模型验证

核心算法——LLM提示构造

上下文提示遵循的上下文与指令集格式

- 省略与人相关的部分内容，如引言、总结、附加属性：不要对答案进行解释。不要提供文中未提及的信息。
- 生成 JSON 输出 将答案以 JSON 格式呈现并按照提示中指定的属性进行呈现。
- 代码格式化：始终将代码以纯文本形式呈现即不要使用 Markdown 格式。
- 纠正错误时需调整答案：
如果结果中存在已指出的错误，应仔细阅读这些错误并尝试修改答案，而不能只是重复原答案。

```
Return a grammar (Xtext) for a DSL and a name and a description of
this DSL

The grammar should encapsulate the following properties:

A DSL for defining an origami tutorial. An origamist can define
their tutorial with name, authors, paper and steps, where a
step has a description and folds to be made. The different
kinds of folds are defined in the Yoshizawa-Randlett system and
they have to follow the rules defined with the Huzita-Hatori
axioms.

Output the grammar in a 'grammar' property, and the name in a 'name'
property, the description in a 'description' property
```

针对 DSL 类型、属性输入以及无基础条件的提示词

算法详细内容

- 输入：交互配置 + 用户输入
- 数据输出：结构化提示（JSON强制格式）

四段式模板

- 指令(Introduction): 固定任务目标
- 上下文(Context): 基版本语法/实例（若存在）
- 输入数据(Input Data): 用户提供的属性/示例/错误消息
- 输出指示(Output Indicator): 严格指定JSON键
(grammar/name/description)

关键优化

- 去冗余指令：
禁用解释性文本（Don't justify answers）
- 差异化修复：
错误修复时强制变更输出（CHANGE the result）

步骤1: 初始语法生成与验证

- 用户或系统触发语法创建，由LLM生成Xtext格式的语法定义。

- DSL Assistant调用Xtext编译器进行验证：

如果语法正确，编译成功，生成Ecore元模型，并存储为有效版本。

如果语法错误，编译失败，Xtext返回详细的错误消息（包括错误类型、位置和描述）。

- 错误分为两类：

编译时错误：如语法错误（无效符号、规则冲突）、链接错误（未解析的引用）。

运行时错误：如文件缺失或空指针异常。

步骤2: 错误反馈与修复提示当验证失败时，更新提示词：

- 当前语法定义：有问题的Xtext语法文本。

- 错误消息：Xtext提供的具体错误描述

- （例如，Error raised during compiling of Xtext grammar XtextSyntaxDiagnostic:

null:2 required(...) + loop did not match anything at input '<EOF>') 。

- 修复指令：GPT-4o被明确指示“仔细阅读错误，尝试更改结果，不要直接返回相同语法”。

输出格式强制为JSON，包含修正后的语法、名称和描述。

- 上下文管理：如果当前版本有会话线程（例如，基于先前对话），提示会继承上下文；否则，创建新线程以确保连贯性。

步骤3: LLM生成修正后的版本:

识别错误根源（如缺失规则或符号不匹配），并应用领域知识调整语法。

输出为JSON结构，确保语法文本为纯文本格式（避免Markdown）。

关键行为：模型被训练为“主动更改结果”，避免简单复制粘贴，以提高修复质量。

步骤4: 迭代验证与终止修正后的语法再次通过Xtext验证:

如果成功，存储为新版本，并终止修复循环。

如果失败，重复步骤2-3（最多4次尝试）。

终止条件：修复成功、达到最大尝试次数（4次）或超时。

失败时，工具记录错误，但允许用户手动干预。

```
Something went wrong, this is the error:
```

```
Error raised during compiling of Xtext grammar  
XtextSyntaxDiagnostic: null:2 required (...) + loop did not match  
anything at input '<EOF>'
```

```
Carefully read error and try to find and solve the mistake and  
return the new corrected result
```

```
Output the grammar in a 'grammar' property, and the name in a 'name'  
' property, the description in a 'description' property
```

```
Return how the new result is corrected in an 'adjustment' property
```

包含错误消息输入和错误的 DSL 版本
作为基础条件而构建的提示

实验过程

通过4名人类（均为丹麦计算机科学硕士生，具备建模课程背景）

使用DSL Assistant解决17个基于国际象棋DSL的任务。

采用“思考-表达”协议和后续访谈收集数据，重点关注用户操作行为、工具辅助能力和可用性问题的。

•用户操作表现：

- 所有主题均能高效导航和处理数据（如创建/更新DSL语法、生成示例）。
- DSL Assistant成功支持非线性迭代过程：主题可自由回溯到旧版本。
- 在17项任务中，主题主要依赖LLM自动生成内容，但在特定场景仍保留手动控制权，体现工具灵活性。

•工具辅助效果：

- **自动生成能力**：主题成功通过自然语言提示生成DSL语法和示例（成功率100%）。
- **自动修复功能**：所有主题报告修复机制有效。
- **认知负担问题**：主题指出UI元素位置不直观，响应时间较长（平均5-10秒/操作），以及概念混淆

•访谈洞见：

- **语义理解**：主题认为工具能准确捕捉领域意图，但通用符号（如冒号、花括号）降低了领域特异性。
- **功能改进需求**：主题期望添加排序/过滤版本功能，并优化元模型可视化交互性。
- **整体体验**：所有主题认为DSL Assistant对建模课程有辅助价值但强调需进一步简化UI以降低认知负荷。

实验结果——LLM的语法生成和修复能力

实验过程

自动测试GPT-4o生成和修复Xtext语法的能力，覆盖18个领域（12个LLM生成描述、6个人工描述）。每个领域生成12个语法实例（总计216个），评估首次尝试成功率，并对错误语法进行自动修复测试。语法正确性通过Xtext编译时/运行时验证和手动语义分析判定。

首次生成成功率：

- 总体首次尝试成功率为50% (108/216)。

人工描述领域成功率更高（**56%** vs. LLM描述**47%**）表明描述质量显著影响输出。

- 领域间差异显著：

Agricultural Management领域成功率为0%，
Urban Planning和Renewable Energy Management达100%。

- 语义分析：

GPT-4o成功捕获所有领域的核心结构和关系（如枚举、继承、自引用）。
但行为规则（如“LEGO Assembly Kit”的堆叠逻辑）过于简化
而结构规则（如材料列表）高度通用化（例如常用'Pattern': title=STRING格式）

缺乏领域特定符号

LLM-made Definitions		
Supply Chain Optimization	6	6
Legal Contracts	5	7
Genetic Research	8	4
Mental Health Therapy	1	11
Fashion Design and Manufacturing	9	3
Urban Planning	7	5
Education and Learning Analytics	8	4
Agricultural Management	12	
Environmental Monitoring	2	10
Cybersecurity	7	5
Renewable Energy Management	6	6
Smart Home Automation	8	4
Man-made definitions		
Recipe	10	2
Crossword	4	8
House Plant Management	8	4
Knitting pattern	6	6
LEGO Assembly Kit	7	5
Origami Tutorial	5	7
☐ OK ☐ Faulty		

跨领域的单次语法生成

错误修复效果：

- 自动修复总体成功率达86%（83/96个错误语法修复成功）。
上下文模式（保留对话历史）表现优于无上下文模式
（成功率88% vs. 68%），表明会话连贯性提升修复效率。
- 修复尝试次数平均为 1.6 ± 1.0 1次。
人工描述领域修复更快（平均1.3次 vs. LLM描述1.8次）

描述类型	修复模式	平均尝试次数	成功修复率
LLM生成描述	带上下文修复	1.8 ± 1.06	68%
	无上下文修复	2.0 ± 1.02	50%
人类描述	带上下文修复	1.3 ± 0.77	88%
	无上下文修复	1.4 ± 0.93	71%

DSL语法修复率对比表

实验结果——LLM的语法生成和修复能力

Xtest报告错误类型分布：

编译时错误占主导（96.2%），主要为语法符号错误（XtextSyntaxDiagnostic, 224例）和链接错误（XtextLinkingDiagnostic, 283例）。
运行时错误罕见（16例），多由空指针或文件缺失引发

异常类型	阶段	数量
XtextSyntaxDiagnostic	编译时	224
XtextLinkingDiagnostic	编译时	283
TransformationDiagnostic	编译时	96
FileNotFoundException	运行时	8
IllegalStatementException	运行时	7

小结

GPT-4o在单次尝试中可生成约50%的正确语法，成功捕捉领域核心要素，但行为规则过于简化且符号通用化。修复机制高度有效（86%成功率），尤其依赖上下文连贯性。错误主要由符号误用引发，编译时检测便于自动化修复。

实验结果——示例实例化与语法归纳能力

实验过程

实验测试GPT-4o从语法生成示例（实例化）和从示例归纳语法（归纳）的能力，使用6个公开DSL。实例化评估基于三种描述类型（一般、非技术、技术），归纳评估基于示例集（4个或全部示例）。输出通过手动比对地真值（*ground truth*）验证。

结论

GPT-4o能有效实例化示例（尤其技术描述），但一般描述导致显著偏差。
语法归纳可行但易出错误（58.3%初始错误率），常出现语义不匹配（欠拟合/过拟合）。
归纳质量依赖示例集完整性，需结合用户反馈迭代优化。

实验结果——示例实例化与语法归纳能力

示例实例化效果：

● 总体表现：

生成的示例正确使用关键词和数据规则

如 “Coffee Machine FSM” 的state和transition

但存在小偏差（如多余括号、错误大小写）。

技术描述（明确引用语法规则）最接近地真值

（偏差率<5%）

而一般描述

如 “A Coffee Machine FSM that can make coffee and tea”

常遗漏数据或添加幻觉内容

● 案例展示：

对比 “Coffee Machine DSL” 的原始示例

与三种描述生成的实例。

技术描述几乎复原地真值

非技术描述简化了逻辑（如省略超时处理）

一般描述则缺乏细节

Ground truth [20]

```
machine CoffeeMachine [  
  initial ^initial  
  state ^initial [  
    on input "coin" output "what drink do you want?" and go to selection  
    on input "break" output "machine is broken!" and go to broken  
  ]  
  state selection [  
    on input "tea" output "serving tea" and go to brewingTea  
    on input "coffee" output "serving coffee" and go to brewingCoffee  
    on input "timeout" output "coin returned; insert coin" and go to ^initial  
    on input "break" output "machine is broken!" and go to broken  
  ]  
  state brewingCoffee [  
    on input "done" output "coffee served. Enjoy!" and go to ^initial  
    on input "break" output "machine is broken!" and go to broken  
  ]  
  state brewingTea [  
    on input "done" output "tea served. Enjoy!" and go to ^initial  
    on input "break" output "machine is broken!" and go to broken  
  ]  
  state broken  
]
```

Technical: A FSM called CoffeeMachine FSM. CoffeeMachine has five states, initial, selection, brewingCoffee, brewingTea, and broken. CoffeeMachine's initial state is initial. The initial state has three transitions: 1) with input "coin" and output "what drink do you want?" and the target is the selection state, 2) with input "break" and output "machine is broken!" and the target is the broken state...

```
machine CoffeeMachine[  
  state ^initial[  
    on input coin output "what drink do you want?" and go to selection  
    on input break output "machine is broken!" and go to broken  
  ]  
  initial ^initial  
  state selection[  
    on input tea output "serving tea" and go to brewingTea  
    on input coffee output "serving coffee" and go to brewingCoffee  
    on input timeout output "coin returned; insert coin" and go to ^initial  
    on input break output "machine is broken!" and go to broken  
  ]  
  state brewingCoffee[  
    on input done output "coffee served. Enjoy!" and go to ^initial  
    on input break output "machine is broken!" and go to broken  
  ]  
  state brewingTea[  
    on input done output "tea served. Enjoy!" and go to ^initial  
    on input break output "machine is broken!" and go to broken  
  ]  
  state broken[]  
]
```

General: A Coffee Machine FSM that can make coffee and tea.

```
machine CoffeeMachineFSM [  
  state Idle [  
    on "coin" go to Ready  
  ]  
  state Ready [  
    on "buttonCoffee" go to BrewingCoffee  
    on "buttonTea" go to BrewingTea  
  ]  
  state BrewingCoffee [  
    on "done" output "coffee" and go to Idle  
  ]  
  state BrewingTea [  
    on "done" output "tea" and go to Idle  
  ]  
  initial Idle  
]
```

Non-technical: A coffee machine where one can buy tea and coffee with a coin. Initially, one can insert a coin. After inserting a coin, one can select an beverage, either tea or coffee, if this takes too long it returns the coin. After selection the beverage kind, it will either serve coffee or tea. Anytime during the interaction with the coffee machine, it is possible to break the machine, so it no longer works.

```
machine CoffeeMachine {  
  state Idle {  
    on 'insertCoin' go to InsertCoin  
  }  
  initial Idle  
  state InsertCoin {  
    on 'selectBeverage' go to SelectBeverage  
    on 'timeout' go to Idle  
  }  
  state SelectBeverage {  
    on 'serveCoffee' go to ServeCoffee  
    on 'serveTea' go to ServeTea  
  }  
  state ServeCoffee {}  
  state ServeTea {}  
  state Broken {}  
}
```

咖啡机器制作的不同DSL对比

语法归纳效果：

- **归纳成功率**：从示例归纳的语法中，7/12个初始输出有错误
主要为符号误用，如“*School DSL*”中用花括号替代方括号
修复后，2个错误语法被成功纠正（修复率28.6%）。
- **语义偏差**：归纳常出现过拟合或欠拟合：
 - **欠拟合**：如“*School DSL*”将教师和班级简化为字符串（而非专用类）
丢失原始语义层次。
 - **过拟合**：如“*Robot DSL*”添加未提及的常量（如速度值）
引入冗余规则。
- **归纳基础影响**：使用全部示例归纳的语法比部分示例更健壮（错误率低20%）
但仍依赖后续用户交互优化。

优点

1. 降低DSL开发门槛

DSL Assistant 能够帮助编程经验有限的用户从自然语言或示例生成 DSL 的语法规则，降低了 DSL 开发的技术门槛。

2. 多种交互模式支持非线性开发流程

支持版本管理、回退、错误修复等功能，使用户可以灵活迭代地发展 DSL，而非依赖线性开发过程。

3. 强大的错误修复能力

使用 GPT-4o 自动修复语法错误的成功率达到约 **86%**，对于初学者和非专业程序员尤其有帮助。

4. 语法生成质量总体可接受

在“一次生成”中，约有 **50%** 的 DSL 语法是正确的，尤其在人为编写的描述中成功率更高（56%）。

5. 语法与实例联动，有助于示例驱动建模

工具可从语法实例生成 DSL，或从 DSL 推导出示例，有助于在语言开发中实现语法与实际应用的互动。

6. 用户体验正向反馈

用户测试表明，认为工具可以替代繁琐的 Eclipse IDE 配置，显著提高建模效率。

缺点

1. 生成语法过于通用，缺乏领域特异性

尽管能生成语法，但使用的结构（如冒号、花括号）太通用，难以反映特定领域的表达习惯，行为类规则表现尤为不足。

2. 上下文依赖性较强

没有上下文时，GPT-4o 的表现较差（如修复成功率下降），必须依赖上下文线程来保证生成质量。

3. 示例泛化能力有限

虽然能从示例中泛化出语法，但常出现 **过拟合**（把具体数据做成语法规则）或 **欠拟合**（未提取出关键结构）等问题。

4. 用户界面存在可用性问题

用户反馈界面中版本管理、属性、上下文等概念混淆；提示不够清晰，缺乏工具提示、排序过滤功能，以及语法高亮。

5. 性能受领域影响

某些领域（如“农业管理”）完全未能生成有效语法，显示出在某些 DSL 任务中的不稳定性。

6. 实验结果可重复性和外推性不足

部分结果依赖手动分析和少量人类参与者；未测试其他 LLM 模型或多种语言输入，外部效度有限。

Part04

MoCQ:通过整体神经符号方法自动 检测静态漏洞(2025)

<https://arxiv.org/abs/2504.16057>



北京大学

PEKING UNIVERSITY

北京大学软件与微电子学院
School of Software & Microelectronics

INFOSEC

Laboratory

北京大学信息安全实验室

网络和软件安全保障教育部重点实验室



小组主页: <http://www.pku-exploit.com/>

静态分析方法：

- 静态漏洞检测能够无须运行代码即可分析大规模代码库，并且具有严格的控制流和数据流分析能力已被业界和学术界广泛采用（如 CodeQL、Joern 等）。
- 但传统方法依赖人工设计漏洞模式，工作量巨大且易出现覆盖不全和模式过时等问题（例如撰写规则耗时可达数周）

LLM方法：

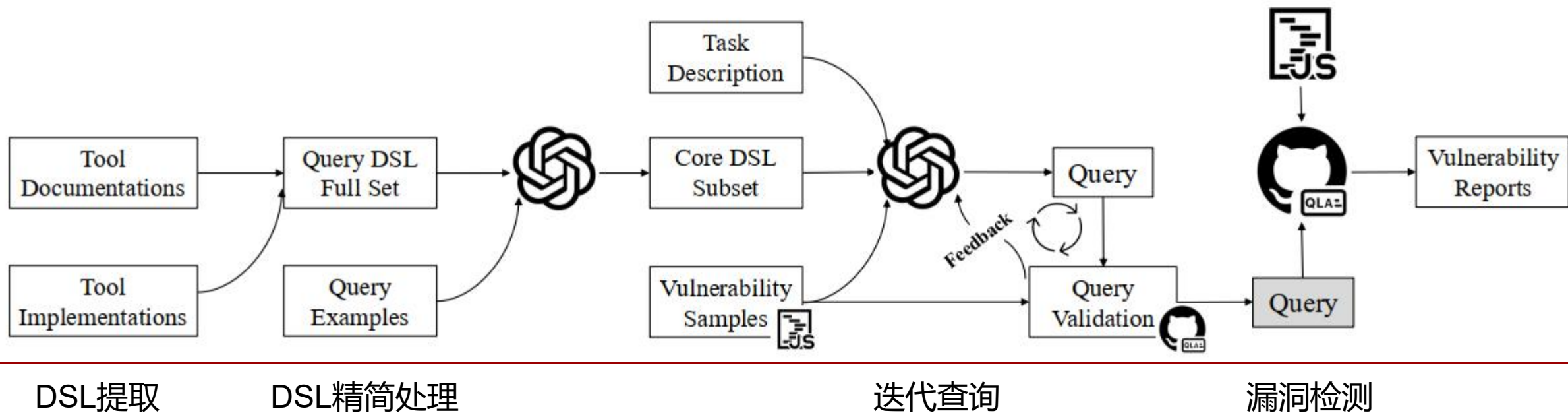
- 近期研究尝试利用大模型（LLM）来自动发现漏洞模式，利用其优秀的模式识别和语义理解能力无需显式手工编写规则。
- 然而，LLM缺乏严格的逻辑推理，输出结果常有不确定性和偏差，高误报率严重且受限于输入长度，在复杂程序上表现欠佳。

动机：

- 由于纯静态分析和纯 LLM 方法各有优缺，单独使用均难以高效、自动地生成高质量漏洞检测查询。
- 论文提出将两者结合的神经符号框架 MoCQ：
由 LLM 自动提取漏洞模式并生成查询语句
再由经典静态分析进行迭代验证和反馈优化，从而兼顾 LLM 的灵活表达和静态分析的严谨性
实现可扩展的自动化漏洞检测。
- MoCQ 正是基于这种理念设计，将 LLM 与符号分析组合起来，期望减少人工参与、挖掘更多潜在漏洞。

方法总览

- (1) MoCQ 首先分析开放知识，例如文档和工具实现，以提取查询 DSL，然后构建核心 DSL 子集（4.1）。
- (2) MoCQ 通过与基于跟踪的符号查询验证器的反馈循环来生成漏洞查询
- (3) 优化和改进查询语句。
- (4) MoCQ 输出生成的查询，该查询可用于检测新的漏洞。



DSL 提取与子集选择：MoCQ 自动抓取静态分析工具的文档（如 Joern 的 “Steps”列表）和源码调用 LLM 摘要各个查询 API 的作用。

提取后发现 DSL 特性冗余度高，大量特性并非生成常见查询所必需。

再通过让 LLM 分析典型查询示例，选择常用的核心 API 子集，保留 DSL 的主要表达能力同时大幅降低复杂度。为了保证语法正确，MoCQ 将最终的 DSL 子集以 BNF 文法形式给 LLM，增强 LLM 对 DSL 结构的理解。

Joern是一个开源的静态分析
根据能够生成代码属性图（CPG）
源码不需要编译也可以分析（适合分析固件、设备）

```
<traversal> ::= <cpg_start> . <step_chain>

<cpg_start> ::= cpg | cpg.method | cpg.call | ...

<step_chain> ::= <step> | <step> . <step_chain>

<step> ::= <filter_step> | <complex_step>

<filter_step> ::= .where( <predicate> )

<complex_step> ::= .filter( <predicate> ) | .reachableBy( <traversal> ) | ...

<predicate> ::= _.name( <reference> ) | <identifier>

<reference> ::= Operators. <operator>

<operator> ::= assignment | arrayAccess | fieldAccess | ...
```

MoCQ提取得到的BNF格式

查询生成过程:

- 针对每个漏洞类型, 首先准备自然语言任务描述 (一般为该漏洞的检测逻辑分步骤说明) 和若干真实示例代码。
MoCQ 使用链式思考提示将任务拆分成步骤, 引导 LLM 分步构造查询。
- 对每个示例, LM 生成一个子查询, 并执行以下流程:
调用 LLMQueryGen 生成查询
使用验证器执行该查询并获取任何错误/程序状态
然后在无错误时对查询进行优化
循环迭代直至成功。
最后将所有子查询合并为单一查询。
- 示例代码还用于检查生成的查询能否正确定位这些已知漏洞
从而提供语义层面的反馈

Algorithm 1: Iterative query generation in a feedback loop.

```
Input : Task-Description  $D^t$ , Vulnerability examples  $E^t$ 
Output: Query  $Q^t$ 
1  $QList \leftarrow []$ 
2 foreach  $e_i^t \in E^t$  do
3    $PState \leftarrow null$ 
4   for  $i \leftarrow 1$  to  $MaxN$  do
5      $Q_i^t \leftarrow LLMQueryGen(e_i^t, D^t, PState)$ 
6      $error, PState \leftarrow Validation(Q_i^t, e_i^t)$ 
7     if not  $error$  then
8        $Q_i^t \leftarrow Optimize(Q_i^t)$ 
9        $QList.add(Q_i^t)$ 
10      break
11    end
12  end
13 end
14  $Q^t \leftarrow \bigcup_{q \in QList} q$  // Merge per-example queries
15 return  $Q^t$ 
16
17 function  $Validation(Q, e)$ :
18    $S \leftarrow null$ 
19   foreach  $B \in Q$  do
20      $S = Execute(B, e, S)$  // Capture fine-grained runtime info
21   end
22   return  $S.err, S.programState$ 
23 end
```

迭代反馈机制：

- 生成查询往往一开始无法完全正确，MoCQ 设计了细粒度的符号验证器来定位问题。
验证器在查询执行时追踪每个代码块的变量状态和中间值（程序状态）
 - 语法验证发现语法错误时，会定位到具体的规则进行反馈；
 - 执行验证发现运行错误（如未定义变量、类型不匹配等）时，会建议相似 API 名称或操作
 - 语义验证则检查查询是否实际匹配到示例漏洞（而非仅仅完整执行）
- 如果失败则仅提供与漏洞路径相关的状态给 LLM，以帮助其找出逻辑缺陷。
每次迭代，LLM 根据这些反馈反思并更新查询（参考自我反射策略）
直到查询正确或达到最大尝试次数

泛化与优化策略：

- 生成的初始查询可能过拟合于示例，限制了对其他代码的检测能力，或者过于宽泛而造成误报。MoCQ 设计了几条启发式规则：
- 检查查询中是否依赖示例中的具体常量/标识符（例如硬编码的变量名和字符串的字面值）或是否包含只适用于特定程序结构的深链式模式，一旦发现过拟合特征就让 LLM 对查询进行泛化（例如抽象掉固定字符串、简化过深的 AST 路径）。
- 对于误报，MoCQ 利用已知的真阳性集，将查询应用于这些代码以外的部分，找到误报位置再反馈给 LLM 让其加入额外条件或修正逻辑，以消除误报。
- 最后，通过 LLM 将各示例查询合并为一个查询，以减少冗余条件、提高执行效率

数据集来源：

选取 OWASP Top10 中的 7 类常见漏洞，分别在 PHP 和 JavaScript 语言的开源项目中评估。

针对每种漏洞类型，筛选 20 个流行项目并收集所有公开已知漏洞（通过 CVE、GitHub issue 等渠道构建“漏洞库”数据集。随机将项目分为两组：15 个项目（含 192 个漏洞）用于查询生成（训练）

5 个项目（73 个漏洞）用于性能评估（测试）

此外，为评估发现新漏洞的能力，还收集了 19 个最新版本项目的代码库，进行实际的“零日漏洞”检测。

工具与模型：

实现时采用静态分析工具 Joern（Scala DSL）为主要执行引擎（支持 PHP/JS）

并选用 CodeQL（SQL DSL，主要支持 JS）进行对比实验

默认 LLM 使用 Claude 3.7 Sonnet 模型，部分消融实验还使用 GPT-4o 验证稳定性。

评估指标：

使用召回率（真正例/总真例）和精确度（真正例/（真正例+假阳性））以及综合的 F1 值评估检测效果

此外还统计查询生成效率（生成每条查询所需时间）、新漏洞发现数、查询复杂度等

性能对比对象包括专家人工编写的规则查询、纯 LLM 方法和其他最新神经符号工具（如 IRIS）等

漏洞检测性能:

- MoCQ 生成的查询在测试集上表现良好，共检测出 56/72 个已知漏洞，召回率 0.77，精确度 0.40。
- 相比之下，专家人工查询的平均召回率 0.70、精度 0.34。
- 纯 LLM 方法（不依赖静态分析）效果较差，精度通常很低
例如表中 “Pure LLM” 平均召回 0.52、精度 0.13，验证了结合静态分析的必要性
- 总体来看，MoCQ 在大多数漏洞类型上性能与专家查询相当，有时甚至更优，并且两者互为补充：
如 PHP XSS、PHP Type 以及 JS 命令注入等场景，MoCQ 捕获了专家遗漏的漏洞。将 MoCQ 查询与专家查询合并使用，可进一步提高检测率，综合召回达 0.87

Vul. Type	# TP _{total}	MoCQ				Experts'				Detection Delta			MoCQ+Experts'				Pure LLM			
		# TP	# FP	Rec.	Prec.	# TP	# FP	Rec.	Prec.	# M∩E	# M\E	# E\M	# TP	# FP	Rec.	Prec.	# TP	# FP	Rec.	Prec.
PHP SQLi	19	16	14	0.84	0.53	17	21	0.89	0.45	15	1	2	18	23	0.95	0.44	9	45	0.47	0.17
PHP XSS	13	8	10	0.62	0.44	7	14	0.54	0.33	7	1	0	8	15	0.62	0.35	9	27	0.69	0.25
PHP Type.	6	4	22	0.67	0.15	3	18	0.50	0.14	3	1	0	4	24	0.67	0.14	2	23	0.33	0.08
PHP Deser.	8	5	4	0.62	0.56	4	6	0.50	0.40	2	3	2	7	7	0.88	0.50	5	57	0.63	0.08
JS Proto.	11	10	8	0.91	0.56	7	12	0.64	0.37	6	4	1	11	13	1.00	0.46	4	39	0.36	0.09
JS Cmd.	5	5	17	1.00	0.23	5	15	1.00	0.25	5	0	0	5	20	1.00	0.20	3	25	0.60	0.11
JS XSS	11	8	15	0.73	0.35	9	11	0.82	0.45	6	2	3	11	12	1.00	0.48	6	34	0.55	0.15
Total / *Avg.	73	56	90	*0.77	*0.40	52	97	*0.70	*0.34	44	12	8	64	114	*0.87	*0.37	38	250	*0.52	*0.13

在测试数据集上使用 Joern 进行的漏洞检测情况

新漏洞发现：

- 在“最新项目”数据集上，MoCQ 自动生成的查询发现了 7 个先前未知的漏洞
涉及 JS 原型污染、命令注入，PHP SQL 注入和 XSS 等
 - 相比之下，官方 Joern 规则和文献中已有专家查询仅发现了 3 个，这 7 个漏洞中有 4 个仅由 MoCQ 新模式检测出
 - 更进一步分析表明，MoCQ 识别出了至少 12 个专家查询未涵盖的新漏洞模式
- 这些新模式往往涉及遗漏的数据流路径或新的易受攻击操作
MoCQ 查询能自动补充相应逻辑，从而成功挖掘零日漏洞。

Project	Vul. Type	Experts'	MoCQ	Contributing New Pattern	Status
forkcms [45]	PHP SQLi	✓	✓	-	Rep.
SyliusCmsPlugin [42]	PHP XSS	×	✓	Ctx-sensitive san. via htmlspecialchars	Rep.
parse-static-imports [46]	JS Cmd.	×	✓	Vulnerable operation with repl	Ack.
es2015-proxy [44]	JS Proto.	×	✓	Data flow via Object.assign	Rep.
web-worker [48]	JS Proto.	✓	✓	-	Rep.
koa-send [49]	JS Proto.	✓	✓	-	Rep.
content-type [43]	JS Proto.	×	✓	Data flow via Reflect.set	Rep.

新发现的漏洞和新的漏洞模式

效率对比:

MoCQ 自动生成查询的效率显著高于人工方式。根据表 3 所示，对于每种漏洞类型，MoCQ 只需几个小时（通常<24h）即可生成查询而专家团队往往需要数天甚至数周多次迭代
例如，JS 原型污染漏洞的自动查询仅需 1.4~1.5 小时/样例，而人工编写此类查询可能耗费 7 周时间。

	PHP SQLi	PHP XSS	PHP Type.	PHP Deser.	JS Proto.	JS Cmd.	JS XSS
MoCQ							
# Examples	23	28	11	13	15	10	13
Time	15.3h	24.6h	5.2h	18.7h	21.4h	7.9h	10.5h
Experts'							
# Commits	5	5	10	17	28	1	1
Time Span	2d	3d	7d	5w	7w	-	-

查询生成的效率

实验结果

消融实验：

系列对比实验验证了关键技术的重要性。

- 关闭 DSL 子集化（使用全 DSL）会使查询效果显著下降，平均 F1 从 0.51 下降到0.33

表明子集选择有助于模型更好地理解查询语言。

- 去除误报消除机制（W/O FP）会显著增加误报率，F1 降至 ~0.30

- 去除泛化步骤（W/O Gen）则导致遗漏更多漏洞，F1 降至 ~0.29

- 对比不同 LLM 模型，Claude 3.7 Sonnet 的总体 F1（0.51）优于 GPT-4o（0.40）

显示前者在生成代码任务上更擅长

- 采用 CodeQL 代替 Joern 进行 JS 分析也做了对比：

虽然 CodeQL 语法更复杂，但在 JS 原型污染上获得了更高 F1（0.72）

在其他 JS 漏洞上略逊于 Joern。

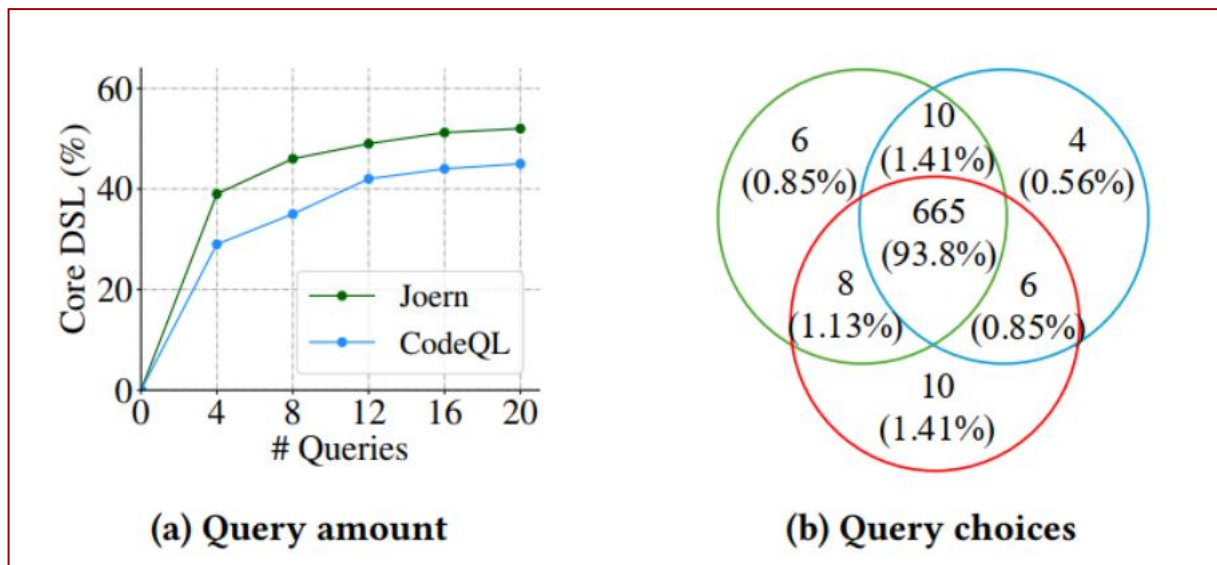
综上，MoCQ 的各项创新（DSL 子集、反馈迭代、泛化与合并等）均对最终性能有显著贡献。

Vul. Type	MoCQ	DSL ₁₀₀	W/O _{FP}	W/O _{Gen}	GPT-4o	CodeQL
PHP SQLi	0.65	0.38	0.54	0.37	0.44	-
PHP XSS	0.52	0.23	0.32	0.25	0.43	-
PHP Type.	0.25	0.15	0.19	0.23	0.31	-
PHP Deser.	0.59	0.41	0.34	0.28	0.50	-
JS Proto.	0.69	0.54	0.33	0.28	0.54	0.72
JS Cmd.	0.37	0.29	0.18	0.32	0.22	0.35
JS XSS	0.48	0.33	0.23	0.32	0.35	0.38
Average	0.51	0.33	0.30	0.29	0.40	0.48

消融实验的F1分数对比

DSL 提取子集性能对比:

- 针对Joern和Codeql进行实验, 随着查询数量增加, 核心子集趋于稳定
- 性能下降明显: 使用完整 DSL 时, 模型更容易生成无效或冗余的查询, 导致有效查询更难生成
误报率更高, 整体检测性能下降约 35%;
- 效率问题严重: DSL全集在多个类型上都触发了“迭代次数上限”,
即无法在规定次数内生成有效查询, 显著拉低了系统的可用性;
- 子集稳定性良好: 多次实验中发现, 使用不同的查询示例选择构建 DSL 子集
最终保留下来的核心 API 子集重叠率达 93.8%, 说明核心 DSL 子集具有较强的通用性 (见图)
- 保留率适中: 最终 DSL 子集大约保留了 45~55% 的 API
- 删除的大多数为冗余功能、调试功能或与当前语言无关的语言特定操作 (如 Joern 中的 address)



对DSL产生影响的查询

神经符号协同新框架：

首次提出将 LLM 与传统静态分析端到端自动化结合，由 LLM 自动生成漏洞查询并通过静态分析进行严谨验证和改进。这种任务划分方式充分利用了 LLM 的模式识别优势和静态工具的可扩展性，实现了“自动模式发现+查询执行”闭环。

DSL 子集化与反馈机制：

创新地设计了 DSL 子集选择技术，让 LLM 专注于 DSL 的核心表达，并使用 BNF 引导确保语法正确。

引入基于程序轨迹的反馈迭代，通过语法验证、运行时追踪和语义检查不断修正查询。

此外，还提出了基于 LLM 的误报剔除和查询泛化策略，进一步提升检测的精确度和鲁棒性。

全自动漏洞检测系统 MoCQ：实现了完整的 MoCQ 平台，支持多种漏洞类型和工具（如 Joern、CodeQL）并通过实验证明其有效性。

MoCQ 在仅用少量示例的情况下快速生成高质量查询，其性能达到了与专家编写查询相当的水平。

实证发现新漏洞：

在实际应用中，MoCQ 生成的查询成功发现了 7 个先前未知漏洞

相比之下，专家查询只能发现其中 3 个。同时，MoCQ 还挖掘出 12 个专家遗漏的漏洞模式。

这些结果表明 MoCQ 能够扩展传统检测模式，具备实际价值。

工程效率提升：

MoCQ 极大减少了漏洞查询的人工设计成本。相较于专家多天工作，MoCQ 可在几小时内生成查询。

同时，MoCQ 也能辅助完善现有查询：

对已有人编写的 JS 原型污染规则进行自动优化后，进一步检测到 3 个漏报漏洞，展示了对已有工具的增补能力

局限和未来工作

依赖示例数据：

当前 MoCQ 需要先验的一小批真实漏洞样本来指导查询生成。

示例选择不当可能影响结果，未来可研究如何利用更多不同视角的示例或自动生成负样本来提升泛化性。

此外，引入人工标注的负例（真实非漏洞代码）有望进一步降低误报。

LLM 本身限制：

虽然 MoCQ 的反馈机制大幅降低了 LLM 的错误输出，但生成过程仍需多次迭代修正。

未来可尝试对 LLM 进行微调或强化学习，将符号验证器作为奖励反馈，以提升模型对 DSL 的掌握程度。

此外，探索更强或更专业化的语言模型也可能带来改进。

跨文件与动态代码：

由于依赖静态代码表示，MoCQ 对跨文件交互（如全局变量、跨文件函数调用）

和动态语言特性（PHP/JS 的动态调用）支持有限，这导致部分漏洞无法检测。

未来可结合跨文件分析和更强的程序表示技术来解决这些问题。

工具支持范围：

目前主要基于 Joern（支持 CPG）和 CodeQL，尚不支持所有语言。

扩展到更多语言和分析引擎是未来方向之一。同样，MoCQ 可以用来增强现有黑盒工具

通过模拟目标工具的输出结果引导查询生成，为闭源工具提供可解释的替代方案。

其他优化方向：

MoCQ 框架可以应用于更多场景，如漏洞克隆检测等

并可探索更智能的提示工程和自动化管道，使其生成的查询更简洁、更高效。

Part05

领域驱动的 LLM 开发：对 RAG 和 微调实践的见解(2024, poster)

<https://dl.acm.org/doi/abs/10.1145/3637528.3671445>



北京大学

PEKING UNIVERSITY

北京大学软件与微电子学院
School of Software & Microelectronics

INFOSEC

Laboratory

北京大学信息安全实验室

网络和软件安全保障教育部重点实验室



小组主页: <http://www.pku-exploit.com/>

领域需求：

通用LLM在特定领域任务中往往缺乏专业知识，容易出现“幻觉”或回答不准确的问题。

RAG作用：

检索增强生成（RAG）通过接入内部知识库，无需重新训练即可扩展模型能力，实现模型在线获取最新领域知识。

微调作用：

模型微调通过在领域专用数据上更新模型参数，使模型系统化地学习通用预训练中未覆盖的新知识，提高对专业术语和任务的理解度。

动机总结：

RAG和微调各自针对不同问题：前者解决知识时效和覆盖度，后者系统化地将领域知识融入模型权重。

RAG（检索增强生成）：

引入外部知识库，通过“检索+生成”两步提高回答准确性。

优点在于能动态获取最新信息，减少“幻觉”

缺点是需要搭建和维护检索管道、构建向量索引，且检索到的信息质量直接影响结果。

RAG方案往往在成本上更低（主要为存储和检索开销），但查询响应速度受检索过程影响。

Fine-Tuning（模型微调）：

对预训练模型进行有监督训练（SFT）和人类反馈优化（RLHF），使其深入学习领域模式和语言风格。

优点是领域适应性强、推理速度快（无需额外检索），能生成高度一致和专业的答案；

缺点是需要大量标注数据和计算资源，更新一次后知识静态化且重新训练成本高。

微调过程技术门槛高，对团队NLP和深度学习经验要求更大。

复杂性与成本对比：

RAG需要构建数据管道和向量库，属于工程实现；

微调则涉及深度学习训练，需高端算力和精细调参。

成本上看，RAG利用现有数据供给LLM，省去了大规模训练开销，通常更为经济；

微调虽然一次性投入高，但在大量相似查询场景中可能降低长期成本（无需每次检索）。

性能上，RAG在处理频繁更新信息场景下表现优势，微调模型在稳定专业领域下表现最佳。

Lab 1 (RAG优化技术) :

实验通过语义切块将文档分段，以提高检索精度并节省查询成本；
引入Graph RAG将知识组织成图结构，更好地捕捉实体关系和层级信息，提高检索相关性；
采用Agentic RAG机制，让LLM智能决定检索策略并迭代优化回答流程，实现自主式知识检索与生成。

Lab 2 (微调流程) :

实验首先利用LLM生成合成数据集以扩充训练样本（例如文本对话或问答对）；
然后进行监督微调（SFT）训练模型，最后可选用RLHF根据人类反馈进一步调优输出质量。
这一流程使模型在领域语料上稳健收敛，并能学习企业特定的问答模式和风格。

Lab 3 (评估与ROI分析) :

实验对RAG和微调模型在标准指标（如准确率、覆盖率、响应时间等）上的表现进行对比
同时评估两者的成本投入及收益。

例如，结果显示混合方案在成本和效果之间取得平衡：
定期微调确保模型语气一致，RAG实时检索最新信息，从而降低总体成本并提高效率

方法选择：

如果任务涉及动态更新、信息量大的领域（如最新法规、实时数据）

优先采用RAG以保持答案时效性

对于知识稳定、可预见的专业场景（如医疗术语、公司内部流程）

采用模型微调可让LLM更深入掌握细节。

成本与技能考量：

RAG实施对工程技术要求更高（数据工程和检索系统搭建）

但一般成本低于微调；微调需要更多标注数据和算力投入，对团队的深度学习经验要求高。

混合使用：

在许多实际场景下可结合RAG与微调：例如对模型进行定期微调以固定基础知识和语气

同时通过RAG检索外部知识库来获取最新信息。这一混合策略可在保证性能的同时控制成本。

应用场景示例：

客服机器人可用RAG实时查阅产品文档，同时微调模型以学习标准回复和公司风格

专业文档问答系统可完全微调以最大化准确

金融或医疗等监管严格行业，结合RAG保证合规、结合微调保证一致性。

报告完毕

谢谢！